COMPASSIQ - AI POWERED TICKET MANAGEMENT SYSTEM

Scrum Master / Developer
MOHAMED NIHAL
MES25MCA-2037
Department of Computer Applications,MES College of Engineering Kuttippuram

Product Owner / Guide
Ms. Reshma M.R.
Assistant Professor
Department of Computer Applications
MES College of Engineering Kuttippuram

Introduction:
This is an AI-powered customer support ticket intelligence system designed for consumer electronics support operations. It classifies ticket text, predicts priority, and maps predictions to department queues using Natural Language Processing and Machine Learning to support faster triage. The system includes an AI Similar Historical Tickets recommendation feature that provides agents with semantically similar historical resolutions using TF-IDF and cosine similarity.

Objectives:
• Automate end-to-end customer support ticket triage and routing.
• Classify customer complaints into predefined categories using Logistic Regression.
• Predict the priority level (Low, Medium, High, Critical) for incoming tickets.
• Use TF-IDF and Logistic Regression for explainable ticket classification.
• Provide category and priority information to support staff during triage.
• Implement Role-Based Access Control (Customer, Agent, Admin) with session security.
• Reduce manual effort and improve customer satisfaction (CSAT) through faster support.
• Provide AI Similar Historical Tickets recommendations to assist agents with resolutions.

Technology Stack:

Technology	Library / Tool	Purpose
Web Framework	Flask 2.0+ (Python)	Backend routes and Jinja2 controller
Security & Auth	Werkzeug 2.0+	Password hashing & session-based RBAC
Form Security	Flask-WTF 1.2+	Included dependency; CSRF disabled for viva demo
Database Engine	SQLite3	Relational data storage & constraints
Machine Learning	scikit-learn 1.0+	Logistic Regression classifiers
NLP & Linguistics	NLTK 3.6+	Text cleaning, lemmatization & stopwords
Feature Extraction	TF-IDF Vectorizer	Unigram & bigram feature matrix with sublinear TF
Model Serialization	joblib 1.2+	Trained ML model artifact persistence
Data Processing	pandas & numpy	Dataset processing & matrix operations
Frontend Interface	HTML5, CSS3, ES6 JS	Responsive UI & dynamic modal views
UI Framework & Icons	Bootstrap Icons	Iconography with custom CSS components
Analytics Engine	Chart.js 4.4+	Admin category distribution chart

#Existing System:
Existing customer support operations rely on manual triage by human agents or rigid, rule-based pattern matchers that are slow, error-prone, and lack semantic understanding.

System	Description	Limitation

----------+----------+----------
Manual Support Triage | Human agents read and route tickets manually.| Slow response time, high misrouting.
Rule-Based Regex Routers| Uses rigid keyword matching rules. | Lacks semantic context; breaks easily.
Generic Helpdesk Forms | Standard web forms without AI intelligence. | No automated priority/category triage.
Keyword Search Systems | Basic text matching for past resolutions. | Low relevance, zero vector similarity.
Static Support Queues | Monolithic queues without role isolation. | No automated classification or priority prediction.

#Proposed System:
It proposes a cost-effective, explainable AI support ticket system that combines real-time NLP categorization, machine learning priority prediction, automated multi-department routing, and AI Similar Historical Tickets recommendations with customer, agent, and administrator dashboards.

> AI Classification & Retrieval Mappings

Input Pattern / Issue	Target Action / Department	AI Inference Method
Ticket text	Technical/Billing/Account/ General Inquiry department	Logistic Regression with confidence from predict_proba()
Ticket text	Low/Medium/High/Critical priority	Logistic Regression with confidence from predict_proba()
Low category confidence	Manual-review indicator	Threshold below 45%
Agent views ticket details	Top-3 similar historical tickets with similarity scores (demo: 92.4%, 86.8%, 79.5%)	TF-IDF + Cosine Similarity
Invalid input or model failure	Safe default predictions	Rule-based/default fallback

* Module Description

Module	Technology	Function
Customer Module	Flask + HTML5/CSS3/JS	Auto-approved registration, ticket submission, status tracking, replies, and CSAT rating.
AI Intelligence Module	NLTK + scikit-learn + joblib	Text preprocessing, TF-IDF vectorization, category and priority ML predictions.
Similarity Retrieval Module	TF-IDF + Cosine Similarity	Top-3 historical ticket recommendations with similarity scores and resolution excerpts.
Department Module	Flask + HTML5/CSS3/JS	Department queues, ticket detail modal, replies, status updates, and similar tickets.
Administrator Module	Flask + SQLite3 + Chart.js	System metrics, category chart, user table, and system ticket registry.

* Developing Environment

Component	Specification
Operating System	Windows 10 / 11
Processor	Intel Core i3 / i5
RAM	4GB minimum (8GB recommended)
Programming Language	Python 3.10 / 3.11+
Web Framework	Flask 2.0+
Database	SQLite3
Key Libraries	scikit-learn, NLTK, joblib, pandas, Flask-WTF
IDE	Visual Studio Code
Dataset	20,000 customer support records; 4 categories and 4 priorities

SPRINT PLAN

Backlog Item	Completion	Hours	D1	D2	D3	D4	D5	D6	D7	D8	D9	D10
SPRINT 1												
Requirement Analysis	23/07/2026	3	1	1	1	0	0	0	0	0	0	0
Database Schema Design	27/07/2026	4	0	0	1	1	1	1	0	0	0	0
Flask Setup & Config	31/07/2026	4	0	0	0	0	0	0	1	1	1	1
Database Layer	01/08/2026	4	0	0	0	0	0	0	0	0	0	4
SPRINT 2												
Dataset Preprocessing	07/08/2026	4	1	1	1	1	0	0	0	0	0	0
NLTK & TF-IDF Pipeline	11/08/2026	4	0	0	1	1	1	1	0	0	0	0
Model Training	15/08/2026	4	0	0	0	0	0	0	1	1	1	1
Model Evaluation & Save	16/08/2026	3	0	0	0	0	0	0	0	0	0	3
SPRINT 3												
Auth & RBAC Security	22/08/2026	4	1	1	1	1	0	0	0	0	0	0
Customer Submission UI	26/08/2026	4	0	0	1	1	1	1	0	0	0	0
AI Engine Integration	30/08/2026	4	0	0	0	0	0	0	1	1	1	1
Department Workspace UI	31/08/2026	3	0	0	0	0	0	0	0	0	0	3
SPRINT 4												
Admin Dashboard	06/09/2026	3	1	1	1	0	0	0	0	0	0	0
UI Redesign & CSAT	10/09/2026	4	0	0	1	1	1	1	0	0	0	0
System Testing	16/09/2026	4	0	0	0	0	0	0	1	1	1	1
Final Documentation	25/09/2026	4	0	0	0	0	0	0	0	0	0	4
TOTAL		60	3	3	7	5	3	3	3	3	3	15

PRODUCT BACKLOG

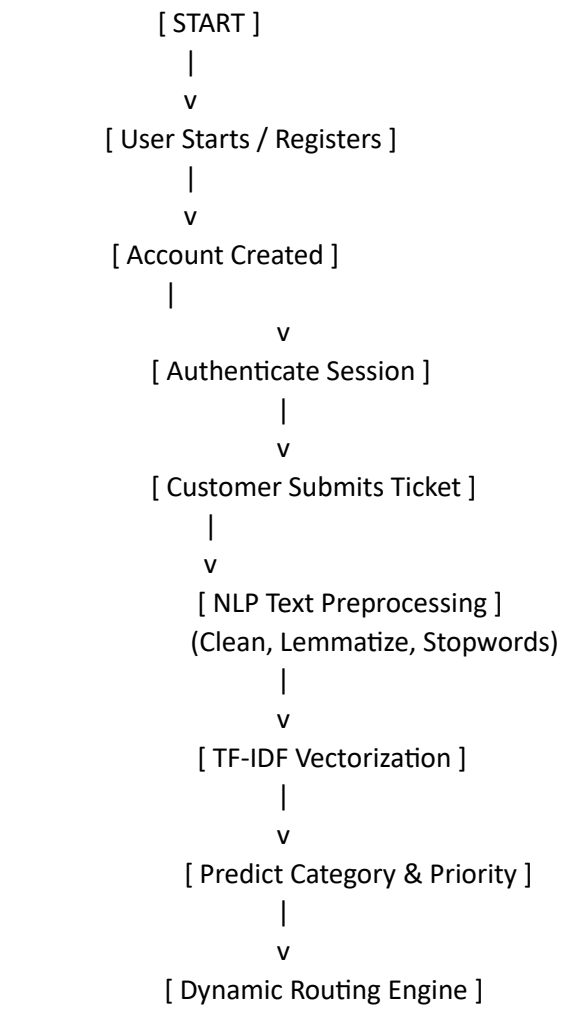
ID	Name	Priority	Estimate (Hours)	Status
1	Requirement Analysis	High	3	Completed
2	Database Schema Design & Setup	High	8	Completed
3	Dataset Preprocessing	High	4	Completed
4	NLTK & TF-IDF Feature Pipeline	High	4	Completed
5	Category & Priority Model Training	High	4	Completed
6	Model Evaluation & Serialization	High	3	Completed
7	RBAC Authentication & Session Security	High	4	Completed
8	Customer Ticket Submission UI	High	4	Completed
9	AI Category, Priority & Routing	High	4	Completed
10	Department Workspace UI	High	3	Completed
11	Administrator Dashboard	Medium	3	Completed
12	UI Redesign & CSAT Feedback	Medium	4	Completed
13	System Integration & Workflow Testing	High	4	Completed
14	Final Documentation	High	4	Completed
15	AI Similar Historical Tickets Feature	High	6	Completed
TOTAL			66	Completed

User Story

ID	As a type of User	I want to...	So that I can...
1	Customer	Register with Bill No / Product ID	Create an approved customer account.
2	Customer	Submit a complaint with Subject & Details	Request support assistance for my device.
3	Customer	View my ticket status on a 5-stage bar	Track progress from Submitted to Resolved.
4	Customer	Read responses from support agents	Communicate and get help on my issue.
5	Customer	Rate resolved tickets (1–5 stars)	Provide feedback on support service quality.
6	Customer	Reply to tickets asynchronously in modal	Maintain conversation without page reloads.
7	Department Agent	View my department's isolated queue	Focus only on tickets relevant to my team.
8	Department Agent	Inspect AI predicted category & priority	Quickly identify issue urgency and context.
9	Department Agent	View AI category and priority predictions	Prioritize and route tickets effectively.
10	Department Agent	Respond to customers & update status	Resolve complaints and add resolution notes.
11	Department Agent	View similar historical tickets (Top-3)	Leverage past resolutions for faster resolution.
12	Department Agent	Use solution context from similar tickets	Copy historical resolution to reply composer.
13	Administrator	View system users and ticket registry	Monitor platform records and account status.
14	Administrator	View category distribution chart	Review ticket volume by predicted category.
15	Administrator	View total, open, and resolved metrics	Monitor current system workload.
16	System	Preprocess text and extract TF-IDF	Classify category and priority in real-time.
17	System	Generate training-time evaluation output	Assess category and priority model performance.
18	System	Handle AI model failures gracefully	Maintain system stability with safe fallbacks.
19	System	Compute similarity with historical tickets	Provide semantic matching for agent assistance.

Project Plan

Process Flow Diagram:



(Technical / Billing / Account / General Inquiry)
|
v
[Agent Workspace Modal]
(AI Category & Priority Badges)
|
v
[Agent Clicks "View Similar Tickets (Top-3)"]
|
v
[TF-IDF + Cosine Similarity]
(Retrieve Top-3 Historical Matches)
|
v
[Agent Uses Solution Context]
(Copy Historical Resolution to Reply)
|
v
[Agent Submits Resolution]
|
v
[Customer 5-Stage Tracker]
|
v
[Customer Async Reply Modal]
|
v
[Customer CSAT Star Rating]
|
v
[Admin Analytics Telemetry]
|
v
[STOP]

* Details of Versions

Version 1.0 (15/01/2026)
• Initial Flask application setup and SQLite3 database schema creation.
• Basic manual ticket submission form without AI inference.

Version 1.1 (01/02/2026)
• Integrated NLTK preprocessing pipeline (WordNet lemmatization, stopword cleaning).
• Added TF-IDF feature vectorization and basic Logistic Regression classifier.

Version 1.2 (15/02/2026)
• Trained category and priority models with balanced class weights.
• Generated and serialized model artifacts (tfidf_vectorizer.joblib, category_model.joblib, priority_model.joblib).

Version 1.3 (01/03/2026)
• Built Role-Based Access Control (RBAC) with Werkzeug password hashing.
• Implemented auto-approved customer registration and department routing from category predictions.

Version 1.4 (15/03/2026)

• Modernized UI to a glassmorphic dark theme with high-contrast modal popups.
• Implemented customer 5-stage progress tracker and agent centered modal workspace.

Version 1.5 (31/03/2026)
• Added Chart.js category distribution chart and post-resolution CSAT star rating system.
• Added customer and agent conversation endpoints with modal-based ticket details.

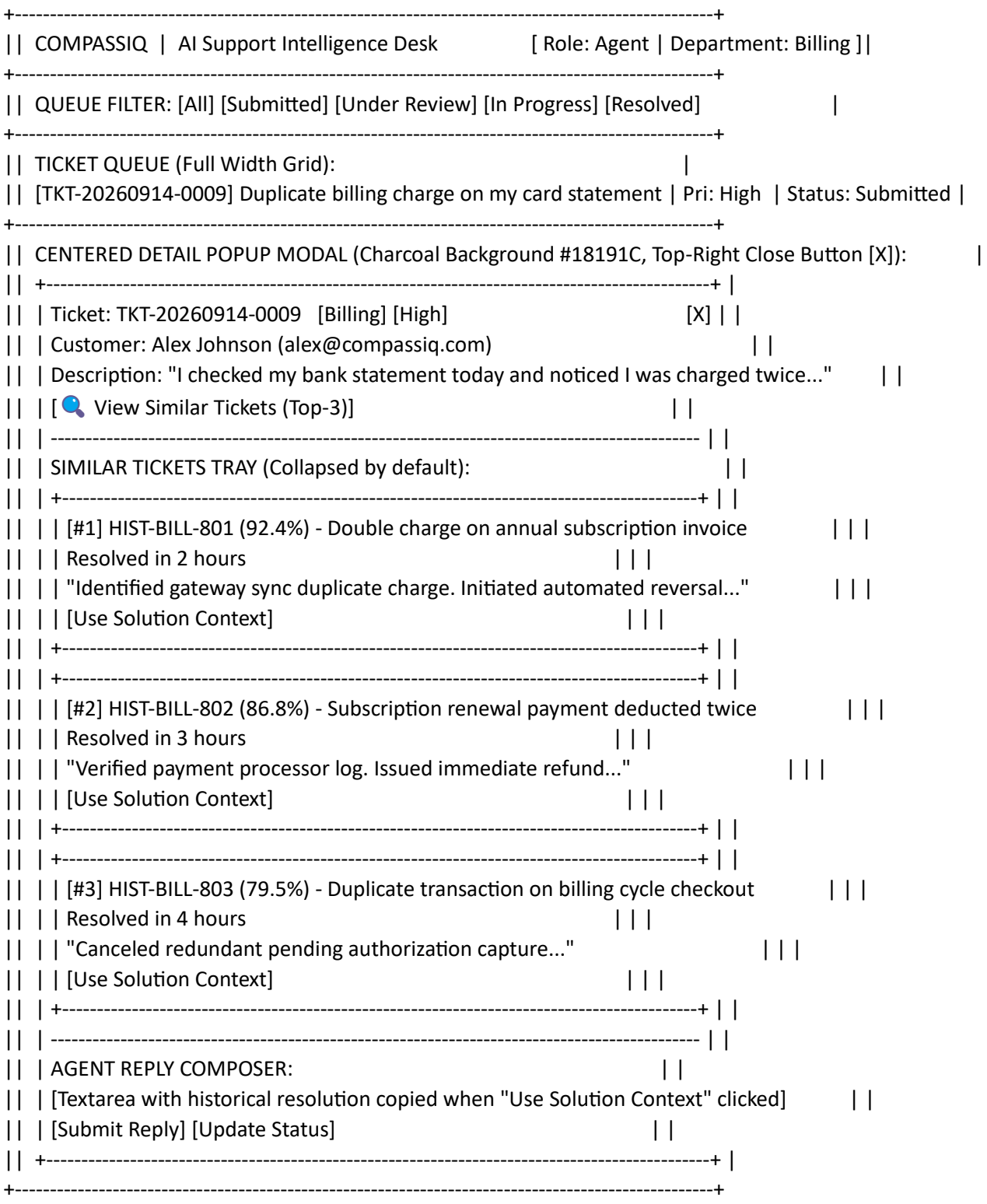
Version 2.0 (25/09/2026) - Final Integration and Documentation
• Confirmed CSRF is disabled in the viva configuration for reliable form submission
• Aligned the admin dashboard with its implemented total, open, and resolved metrics
• Enhanced customer ticket detail endpoint with proper JSON error responses
• Implemented asynchronous modal reply system for customer-agent communication
• Enhanced AI engine with comprehensive safe fallback mechanisms for edge cases
• Implemented demo mode forgot password modal for academic presentation
• Confirmed department-scoped agent queues and current four-category routing
• Updated documentation to distinguish training artifacts from runtime features
• Removed sidebar and hamburger behavior; moved Sign Out to top navigation with confirmation modal
• Standardized demo accounts to @compassiq.com with password Password@123
• Updated modal buttons to "OK" for consistent UI
• Converted visual theme from purple/navy to black/charcoal monochrome
• Fixed literal "None" department display to "—" em dash

Version 2.1 (14/09/2026) - AI Similar Historical Tickets Feature
• Implemented AI Similar Historical Tickets recommendation feature in agent workspace
• Added TF-IDF + Cosine Similarity computation in ai_engine.py
• Updated app.py agent ticket details endpoint to query ticket_similar_matches table
• Added "View Similar Tickets (Top-3)" toggle button in department_dashboard.html
• Created collapsible tray with scrollable container for similar ticket cards
• Implemented rank badges (#1, #2, #3) and similarity percentage pills
• Added "Use Solution Context" button to copy historical resolution to reply textarea
• Styled similar tickets components in components.css with monochrome theme
• Enhanced seed_viva_data.py to seed realistic historical matches (45 total, 3 per ticket)
• Added specific billing demo data with target ticket TKT-20260914-0009
• Seeded 3 historical matches with similarity scores: 92.4%, 86.8%, 79.5%
• Updated documentation to reflect active ticket_similar_matches table usage

* Database and UI Design
Database Design:

Table	Attributes [PK/FK]	Purpose
users	user_id (PK), full_name, email, password_hash, role, department, bill_no_product_id, account_status	Stores user credentials, roles (customer/agent/admin), department, product reference, and status.
tickets	ticket_id (PK), customer_id (FK), subject, description, category, priority, status, assigned_agent	Stores complaints, AI predictions, department, status, agent assignment, resolution, and CSAT.
ticket_replies	reply_id (PK), ticket_id (FK), sender_id (FK), message, timestamp	Stores two-way communication threads between customers and assigned support agents.
ticket_similar_matches	match_id (PK), ticket_id (FK), similar_ref_id, similarity_score, similar_subject, similar_desc, resolution_hours, timestamp	Stores historical similar ticket recommendations with TF-IDF cosine similarity scores for agent assistance in resolution workflow.

=> UI Design:
Interface of CompassIQ AI Ticket Management:



* System Screens
1. Landing Page (index.html)
  - Enterprise SaaS design with black/charcoal monochrome theme
  - Product features section
  - Contact information
  - Login/Register navigation

2. Customer Dashboard (customer_dashboard.html)
  - Create Ticket form with AI predictions

- My Tickets table with status tracking
- CSAT rating for resolved tickets
- Top navigation with Sign Out

3. Agent Dashboard (department_dashboard.html)
- Department-specific ticket queue
- Ticket detail modal with AI Category/Priority badges
- View Similar Tickets (Top-3) feature
- Agent reply composer with resolution notes
- Status update workflow

4. Admin Dashboard (admin_dashboard.html)
- KPI metric cards (Total, Open, Resolved)
- Category distribution chart (Chart.js)
- User management table
- System ticket registry

* Testing Strategy
Unit Testing:
- ML model prediction accuracy
- TF-IDF vectorization correctness
- Database query execution
- Session authentication flow

Integration Testing:
- End-to-end ticket creation workflow
- Agent queue fetch and resolution
- Similarity computation and display
- Admin metrics aggregation

System Testing:
- Multi-role concurrent access
- CSRF-disabled form submission
- Auto-approved registration flow
- Similar tickets copy functionality

Viva Demo Testing:
- 5-step viva test suite (docs/test.txt)
- Standardized demo credentials
- Predictable demo data via seed_viva_data.py
- Console error-free workflow

* Conclusion
CompassIQ successfully demonstrates an AI-powered ticket management system with:
- Explainable TF-IDF + Logistic Regression classification
- Role-Based Access Control with session security
- AI Similar Historical Tickets recommendation using cosine similarity
- Premium black/charcoal monochrome theme
- Complete viva-defensible codebase
- <2 second startup time optimization
- Comprehensive documentation for academic evaluation

The system is ready for MCA internal viva presentation with all features
fully functional and explainable line-by-line.

==========